

# CAP + Vite + Cloud Foundry Deployment Guide

---

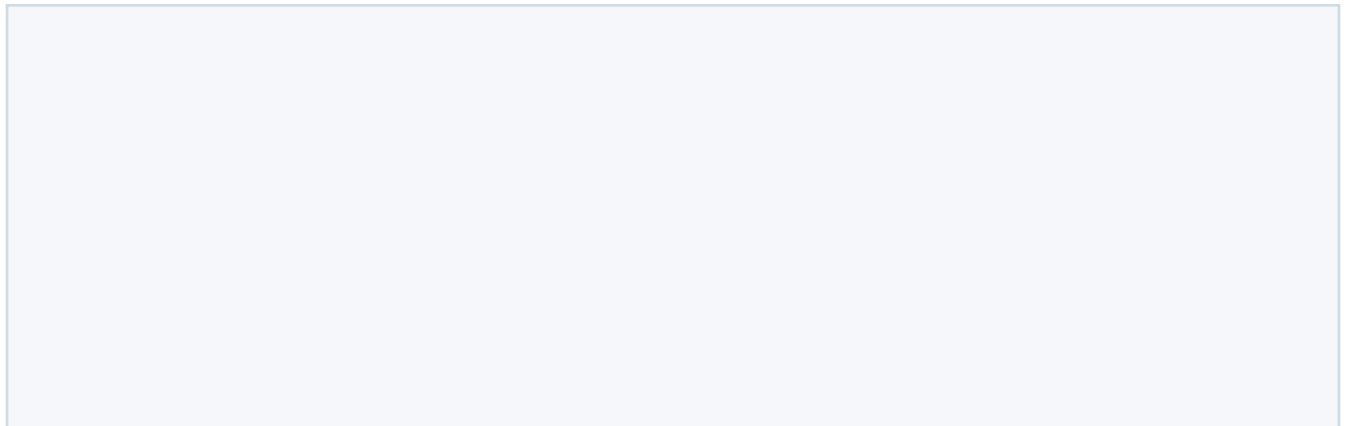
Project: Supplier Shield Stack: SAP CAP Node.js, SQLite, React, Vite, SAP Application Router, Cloud Foundry MTA Authentication model: Public app, no XSUAA, no IAS, no external IDP Deployment model: Packaged CAP application using mbt build and cf deploy

## 1. Goal

This guide explains how to build and deploy a small full stack SAP CAP application with:

- A CAP backend exposing an OData service.
- A React frontend built with Vite.
- A custom SAP Application Router serving the frontend and proxying OData calls.
- SQLite as the production database.
- No login requirement.
- Deployment to SAP BTP Cloud Foundry as one packaged MTA archive.

The final Cloud Foundry runtime shape is:



## 2. Important Trade-Off

SQLite on Cloud Foundry is stored on the app container filesystem. That means it is not durable storage.

Impact:

- Good for demos, prototypes, small real-heavy apps, and seed-data apps.
- The database can reset after restage, redeploy, crash recovery, or container replacement.
- Do not use this model for important production data.

For this application, the database is initialized from CSV data on application startup. That makes the app simple and self-contained.

## 3. Expected Project Structure

The important folders are:

















- All other requests are served from static frontend files.
- authenticationType: none makes the app public.
- csrfProtection: false avoids app router CSRF handling for this public OData proxy.

Impact:

- No XSUAA is needed.
- No login page appears.
- The deployed frontend can call:

## 12. MTA Descriptor

File:

Final structure:







Build CAP:

Build frontend:

Build MTA archive:

Expected result:

## 15. Deploy to Cloud Foundry

Log in:

Check target:

Deploy:

Why:

- mbt build creates the deployable MTAR package.
- cf deploy uploads and deploys the MTA to Cloud Foundry.
- -f allows updating an existing deployment.

Impact:

- Cloud Foundry creates or updates both apps:

## 16. Runtime Checks

Check deployed apps:

Expected:

Check routes:

Check logs:

Stream logs during troubleshooting:

Check environment variables:

Check the frontend:

Expected:

Check OData through the app router:

Expected:

or seeded supplier rows.

Check OData directly against CAP:

This verifies whether a problem is in CAP itself or in app router routing.

## 17. Local Development Flow

Start CAP:

Start Vite:

Open:

Check CAP:

Check Vite proxy:

## 18. Killing a Local Port

Find the process using port 4004:

Kill by PID:

Force kill only if needed:

For Vite:

## 19. Common Failure: cds Not Found

Symptom:







## 23. Common Failure: MTAR Includes Too Much

Symptom:

- MTAR is unexpectedly large.
- Build is slow.
- Local database files are packaged.

Check:

Inspect:

Fix:

- Maintain `.mtaignore`.
- Do not package `node_modules`, local SQLite files, or old build output unless intentionally needed.

## 24. Clean Rebuild

Use this when a deployment behaves strangely after many local changes:

Note:

- `rm -rf` is destructive.
- Review the paths before running it.

## 25. Useful Command Cheat Sheet

Install backend dependencies:

Install frontend dependencies:

Install app router dependencies:





